

Diagramas UML - Proyecto PropTech (Insignia Inmo)

Documento con todos los diagramas de clases del proyecto, incluyendo código PlantUML listo para copiar a herramientas como PlantUML Online, VS Code (plugin), draw.io, etc.

Índice de Diagramas

1. [Diagrama General de Paquetes](#)
2. [Estructuras de Datos](#)
3. [Capa Modelo](#)
4. [Capa DAO](#)
5. [Capa Servicio](#)
6. [Main + Utilidades](#)
7. [Diagrama de Relaciones Completo](#)

1. Diagrama General de Paquetes

```
com.proptech
├─ (Main.java)
├─ modelo          → 8 clases (POJOs)
├─ dao             → 8 clases (SQLite)
├─ servicio        → 13 clases (negocio)
└─ utilidades
    ├─ HashUtils.java
    └─ estructuras → 12 clases (EDs)
```

```
@startuml
    Paquetes
    package "com.proptech" {
        class Main
        package "modelo" {
            [Alerta] [Asesor] [Cliente] [Inmueble]
            [Operacion] [Rol] [Usuario] [Visita]
        }
        package "dao" {
            [ConexionDB] [InmuebleDAO] [ClienteDAO] [AsesorDAO]
            [UsuarioDAO] [OperacionDAO] [VisitaDAO] [FavoritoDAO]
        }
        package "servicio" {
            [InventarioInmueblesService] [ClientesService] [AuthService]
            [OperacionesService] [VisitaService] [AlertasService]
            [GestorVisitasService] [RecomendacionService]
            [AnalisisRelacionesService] [HistorialCambiosService]
            [HistorialYFavoritosService] [DetectorAnomaliesService]
            [ReporteService]
        }
        package "utilidades.estructuras" {
            [Nodo] [ListaEnlazada] [Pila] [Cola]
            [NodoPrioridad] [ColaPrioridad] [NodoArbol]
            [ArbolBinarioBusqueda] [EntradaHash] [TablaHash]
        }
    }
```

```

    [Vertice] [Grafo]
}
package "utilidades" {
    [HashUtils]
}
}

Main --> "modelo" : usa
Main --> "servicio" : usa
Main --> "dao" : usa
Main --> "utilidades" : usa
"servicio" --> "modelo" : manipula
"servicio" --> "utilidades.estructuras" : implementa
"servicio" --> "dao" : persiste
"dao" --> "modelo" : mapea
"dao" --> ConexionDB
@enduml

```

2. Estructuras de Datos

Todas las estructuras son **genéricas** y se implementaron desde cero.

2.1 Nodo y ListaEnlazada

```

@startuml ListaEnlazada
class Nodo<T> {
    - dato: T
    - siguiente: Nodo<T>
    + Nodo(dato: T)
    + getDato(): T
    + setDato(dato: T): void
    + getSiguiente(): Nodo<T>
    + setSiguiente(siguiente: Nodo<T>): void
}

class ListaEnlazada<T> {
    - cabeza: Nodo<T>
    + ListaEnlazada()
    + agregar(dato: T): void
    + get(indice: int): T
    + remove(indice: int): T
    + size(): int
    + imprimir(): void
}

ListaEnlazada o--> Nodo : cabeza
@enduml

```

2.2 Pila

```

@startuml Pila
class Pila<T> {
    - cima: Nodo<T>
    + Pila()
    + push(dato: T): void
    + pop(): T
    + peek(): T
    + estaVacia(): boolean
}

Pila o--> Nodo : cima
@enduml

```

2.3 Cola

```

@startuml Cola
class Cola<T> {
    - frente: Nodo<T>
    - final: Nodo<T>
    + Cola()
    + enqueue(dato: T): void
    + dequeue(): T
    + estaVacia(): boolean
}

Cola o--> Nodo : frente
Cola o--> Nodo : final
@enduml

```

2.4 ColaPrioridad

```

@startuml ColaPrioridad
class NodoPrioridad<T> {
    - dato: T
    - prioridad: int
    - siguiente: NodoPrioridad<T>
    + NodoPrioridad(dato: T, prioridad: int)
    + getDato(): T
    + getPrioridad(): int
    + getSiguiente(): NodoPrioridad<T>
    + setSiguiente(siguiente: NodoPrioridad<T>): void
}

class ColaPrioridad<T> {
    - frente: NodoPrioridad<T>
    + ColaPrioridad()
    + enqueue(dato: T, prioridad: int): void
    + dequeue(): T
    + estaVacia(): boolean
}

```

```

}

ColaPrioridad o--> NodoPrioridad : frente
@enduml

```

2.5 ArbolBinarioBusqueda

```

@startuml ArbolBinarioBusqueda
class NodoArbol<K, V> {
    - key: K
    - value: V
    - izquierdo: NodoArbol<K, V>
    - derecho: NodoArbol<K, V>
    + NodoArbol(key: K, value: V)
    + getKey(): K
    + getValue(): V
    + setValue(value: V): void
    + getIzquierdo(): NodoArbol<K, V>
    + setIzquierdo(izquierdo: NodoArbol<K, V>): void
    + getDerecho(): NodoArbol<K, V>
    + setDerecho(derecho: NodoArbol<K, V>): void
}

class ArbolBinarioBusqueda<K, V> {
    - raiz: NodoArbol<K, V>
    + ArbolBinarioBusqueda()
    + insertar(key: K, value: V): void
    + inorden(resultado: ListaEnlazada<V>): void
    + buscar(key: K): V
}

ArbolBinarioBusqueda o--> NodoArbol : raiz
@enduml

```

2.6 TablaHash

```

@startuml TablaHash
class EntradaHash<K, V> {
    - key: K
    - value: V
    - siguiente: EntradaHash<K, V>
    + EntradaHash(key: K, value: V)
    + getKey(): K
    + getValue(): V
    + setValue(value: V): void
    + getSiguiente(): EntradaHash<K, V>
    + setSiguiente(siguiente: EntradaHash<K, V>): void
}

```

```

class TablaHash<K, V> {
    - capacidad: int
    - tamaño: int
    - tabla: EntradaHash<K, V>[]
    + TablaHash()
    + put(key: K, value: V): void
    + get(key: K): V
    + remove(key: K): V
    + containsKey(key: K): boolean
    + size(): int
    + valores(): ListaEnlazada<V>
    + keys(): ListaEnlazada<K>
    - hash(key: K): int
}

TablaHash o--> EntradaHash : "tabla[]\n(encadenamiento)"
TablaHash ..> ListaEnlazada : retorna
@enduml

```

2.7 Grafo

```

@startuml Grafo
class Vertice<T> {
    - dato: T
    - adyacentes: ListaEnlazada<T>
    + Vertice(dato: T)
    + getDato(): T
    + getAdyacentes(): ListaEnlazada<T>
    + agregarAdyacente(destino: T): void
}

class Grafo<T> {
    - vertices: ListaEnlazada<Vertice<T>>
    + Grafo()
    + addVertex(dato: T): void
    + addEdge(origen: T, destino: T): void
    + getAdyacentes(dato: T): ListaEnlazada<T>
    + printRelations(): void
}

Grafo o--> Vertice : "vertices"
Vertice ..> ListaEnlazada : adyacentes
@enduml

```

3. Capa Modelo

3.1 Inmueble

```

@startuml Inmueble
class Inmueble {
    - codigo: String
    - tipo: String
    - direccion: String
    - precio: double
    - area: double
    - estado: String
    - habitaciones: int
    - banos: int
    - tieneParqueadero: boolean
    - descripcion: String
    + Inmueble()
    + Inmueble(codigo, tipo, direccion, precio, area, habitaciones, banos, tieneParqueadero,
descripcion)
    + getCodigo(): String
    + setCodigo(codigo: String): void
    + getTipo(): String
    + setTipo(tipo: String): void
    + getDireccion(): String
    + setDireccion(direccion: String): void
    + getPrecio(): double
    + setPrecio(precio: double): void
    + getArea(): double
    + setArea(area: double): void
    + getEstado(): String
    + setEstado(estado: String): void
    + getHabitaciones(): int
    + setHabitaciones(habitaciones: int): void
    + getBanos(): int
    + setBanos(banos: int): void
    + isTieneParqueadero(): boolean
    + setTieneParqueadero(tieneParqueadero: boolean): void
    + getDescripcion(): String
    + setDescripcion(descripcion: String): void
}
@enduml

```

3.2 Cliente

```

@startuml Cliente
class Cliente {
    - identificacion: String
    - nombre: String
    - telefono: String
    - presupuestoMaximo: double
    - email: String
    - historialConsultas: ListaEnlazada<Inmueble>
    - favoritos: ListaEnlazada<Inmueble>
    - tipoInmuebleDeseado: String
}

```

```

- zonasInteres: String
- cantMinHabitaciones: int
+ Cliente()
+ Cliente(identificacion, nombre, telefono, presupuestoMaximo, email)
+ getIdentificacion(): String
+ setIdentificacion(...): void
+ getNombre(): String
+ setNombre(...): void
+ getTelefono(): String
+ setTelefono(...): void
+ getPresupuestoMaximo(): double
+ setPresupuestoMaximo(...): void
+ getEmail(): String
+ setEmail(...): void
+ getHistorialConsultas(): ListaEnlazada<Inmueble>
+ agregarConsulta(inmueble: Inmueble): void
+ getFavoritos(): ListaEnlazada<Inmueble>
+ agregarFavorito(inmueble: Inmueble): void
+ getTipoInmuebleDeseado(): String
+ setTipoInmuebleDeseado(...): void
+ getZonasInteres(): String
+ setZonasInteres(...): void
+ getCantMinHabitaciones(): int
+ setCantMinHabitaciones(...): void
}

```

```

Cliente ..> Inmueble : historialConsultas\ny favoritos
@enduml

```

3.3 Asesor

```

@startuml Asesor
class Asesor {
- idAsesor: String
- nombre: String
- especialidad: String
- negociosCerrados: int
- email: String
- telefono: String
- calificacion: double
+ Asesor(idAsesor, nombre, especialidad, email, telefono)
+ getIdAsesor(): String
+ setIdAsesor(...): void
+ getNombre(): String
+ setNombre(...): void
+ getEspecialidad(): String
+ setEspecialidad(...): void
+ getNegociosCerrados(): int
+ setNegociosCerrados(...): void
+ registrarNegocioExitoso(): void
+ getEmail(): String

```

```

+ setEmail(...): void
+ getTelefono(): String
+ setTelefono(...): void
+ getCalificacion(): double
+ setCalificacion(...): void
}
@enduml

```

3.4 Visita

```

@startuml Visita
class Visita {
    - idVisita: String
    - cliente: Cliente
    - inmueble: Inmueble
    - asesor: Asesor
    - fechaHora: String
    - estado: String
    + Visita(idVisita, cliente, inmueble, asesor, fechaHora)
    + getIdVisita(): String
    + getCliente(): Cliente
    + getInmueble(): Inmueble
    + getAsesor(): Asesor
    + getFechaHora(): String
    + setFechaHora(...): void
    + getEstado(): String
    + setEstado(...): void
}

Visita *--> Cliente
Visita *--> Inmueble
Visita *--> Asesor
@enduml

```

3.5 Operacion

```

@startuml Operacion
class Operacion {
    - idOperacion: String
    - tipo: String
    - inmueble: Inmueble
    - cliente: Cliente
    - asesor: Asesor
    - monto: double
    - fecha: String
    + Operacion()
    + Operacion(idOperacion, tipo, inmueble, cliente, asesor, monto, fecha)
    + getIdOperacion(): String
    + setIdOperacion(...): void
}

```



```

+ getTipo(): String
+ setTipo(...): void
+ getInmueble(): Inmueble
+ setInmueble(...): void
+ getCliente(): Cliente
+ setCliente(...): void
+ getAsesor(): Asesor
+ setAsesor(...): void
+ getMonto(): double
+ setMonto(...): void
+ getFecha(): String
+ setFecha(...): void
}

```

```

Operacion *--> Inmueble
Operacion *--> Cliente
Operacion *--> Asesor
@enduml

```

3.6 Alerta

```

@startuml Alerta
class Alerta {
- idAlerta: String
- mensaje: String
- prioridad: int
- fechaCreacion: String
+ Alerta(idAlerta, mensaje, prioridad, fechaCreacion)
+ getIdAlerta(): String
+ setIdAlerta(...): void
+ getMensaje(): String
+ setMensaje(...): void
+ getPrioridad(): int
+ setPrioridad(...): void
+ getFechaCreacion(): String
+ setFechaCreacion(...): void
}
@enduml

```

3.7 Rol

```

@startuml Rol
class Rol {
+ {static} CLIENTE: String
+ {static} VENDEDOR: String
+ {static} ADMIN: String
+ {static} GERENTE: String
- idRol: int
- nombre: String

```

```

+ Rol()
+ Rol(idRol, nombre)
+ getIdRol(): int
+ setIdRol(...): void
+ getNombre(): String
+ setNombre(...): void
}
@enduml

```

3.8 Usuario

```

@startuml Usuario
class Usuario {
    - idUsuario: int
    - email: String
    - passwordHash: String
    - rol: Rol
    - nombre: String
    - telefono: String
    - direccion: String
    - intereses: String
    - fotoPerfil: String
    - activo: boolean
+ Usuario()
+ Usuario(idUsuario, email, passwordHash, rol, nombre)
+ getIdUsuario(): int
+ setIdUsuario(...): void
+ getEmail(): String
+ setEmail(...): void
+ getPasswordHash(): String
+ setPasswordHash(...): void
+ getRol(): Rol
+ setRol(...): void
+ getNombre(): String
+ setNombre(...): void
+ getTelefono(): String
+ setTelefono(...): void
+ getDireccion(): String
+ setDireccion(...): void
+ getIntereses(): String
+ setIntereses(...): void
+ getFotoPerfil(): String
+ setFotoPerfil(...): void
+ isActive(): boolean
+ setActivo(...): void
}

Usuario *--> Rol
@enduml

```

3.9 Modelo - Diagrama de Relaciones Completo

```
@startuml ModeloCompleto
package "com.proptech.modelo" {
    class Inmueble
    class Cliente {
        historialConsultas: ListaEnlazada<Inmueble>
        favoritos: ListaEnlazada<Inmueble>
    }
    class Asesor
    class Visita {
        cliente: Cliente
        inmueble: Inmueble
        asesor: Asesor
    }
    class Operacion {
        inmueble: Inmueble
        cliente: Cliente
        asesor: Asesor
    }
    class Alerta
    class Rol {
        + CLIENTE, VENDEDOR, ADMIN, GERENTE
    }
    class Usuario {
        rol: Rol
    }
}

Visita *--> Cliente
Visita *--> Inmueble
Visita *--> Asesor
Operacion *--> Inmueble
Operacion *--> Cliente
Operacion *--> Asesor
Usuario *--> Rol
Cliente ..> Inmueble : "historial / favoritos\n(ListaEnlazada)"
@enduml
```

4. Capa DAO

4.1 ConexionDB

```
@startuml ConexionDB
class ConexionDB {
    - {static} URL: String
    + {static} conectar(): Connection
    + {static} inicializarTablas(): void
}
```

```
}  
@enduml
```

4.2 InmuebleDAO

```
@startuml InmuebleDAO  
class InmuebleDAO {  
    + guardar(inmueble: Inmueble): void  
    + obtenerTodos(): ListaEnlazada<Inmueble>  
    + actualizar(inmueble: Inmueble): void  
    + eliminar(codigo: String): void  
}  
  
InmuebleDAO ..> Inmueble : CRUD  
InmuebleDAO ..> ListaEnlazada : retorna  
@enduml
```

4.3 ClienteDAO

```
@startuml ClienteDAO  
class ClienteDAO {  
    + guardar(cliente: Cliente): void  
    + obtenerPorId(identificacion: String): Cliente  
    + obtenerPorEmail(email: String): Cliente  
    + obtenerTodos(): ListaEnlazada<Cliente>  
    + actualizar(cliente: Cliente): void  
    + eliminar(identificacion: String): void  
}  
  
ClienteDAO ..> Cliente : CRUD  
ClienteDAO ..> ListaEnlazada : retorna  
@enduml
```

4.4 AsesorDAO

```
@startuml AsesorDAO  
class AsesorDAO {  
    + guardar(asesor: Asesor): void  
    + obtenerPorId(idAsesor: String): Asesor  
    + obtenerTodos(): ListaEnlazada<Asesor>  
    + actualizar(asesor: Asesor): void  
    + eliminar(idAsesor: String): void  
}  
  
AsesorDAO ..> Asesor : CRUD  
AsesorDAO ..> ListaEnlazada : retorna  
@enduml
```

4.5 UsuarioDAO

```
@startuml UsuarioDAO
class UsuarioDAO {
    - cacheUsuarios: ConcurrentHashMap<String, Usuario>
    - cacheRoles: ConcurrentHashMap<String, Rol>
    + UsuarioDAO()
    + UsuarioDAO(forzarRecarga: boolean)
    + crear(email, passwordHash, nombre, nombreRol): Usuario
    + actualizar(usuario: Usuario): boolean
    + buscarPorEmail(email: String): Usuario
    + obtenerTodos(): ListaEnlazada<Usuario>
    + obtenerRolPorNombre(nombre: String): Rol
    - inicializarBaseDatosYRoles(): void
    - cargarUsuariosDesdeSQL(): void
    - cargarRolesDesdeSQL(): void
}

UsuarioDAO ..> Usuario : CRUD + cache
UsuarioDAO ..> Rol : consulta
UsuarioDAO ..> ConexionDB : JDBC
@enduml
```

4.6 OperacionDAO

```
@startuml OperacionDAO
class OperacionDAO {
    + guardar(operacion: Operacion): void
    + obtenerPorId(idOperacion: String): Operacion
    + obtenerTodas(): ListaEnlazada<Operacion>
    + obtenerPorTipo(tipo: String): ListaEnlazada<Operacion>
    + obtenerPorAsesor(idAsesor: String): ListaEnlazada<Operacion>
    + obtenerOperacionesPorCliente(idCliente: String): ListaEnlazada<Operacion>
    + actualizar(operacion: Operacion): void
    + eliminar(idOperacion: String): void
}

OperacionDAO ..> Operacion : CRUD
OperacionDAO ..> ConexionDB : JDBC
@enduml
```

4.7 VisitaDAO

```
@startuml VisitaDAO
class VisitaDAO {
    + guardar(visita: Visita): void
    + obtenerPorId(idVisita: String): Visita
    + obtenerTodas(): ListaEnlazada<Visita>
    + obtenerVisitasPorCliente(identificacionCliente: String): ListaEnlazada<Visita>
}
```

```

+ obtenerVisitasPorAsesor(idAsesor: String): ListaEnlazada<Visita>
+ actualizar(visita: Visita): void
+ eliminar(idVisita: String): void
}

```

```

VisitaDAO ..> Visita : CRUD
VisitaDAO ..> ConexionDB : JDBC
@enduml

```

4.8 FavoritoDAO

```

@startuml FavoritoDAO
class FavoritoDAO {
+ agregarFavorito(identificacionCliente: String, codigoInmueble: String): void
+ quitarFavorito(identificacionCliente: String, codigoInmueble: String): void
+ obtenerFavoritos(identificacionCliente: String): ListaEnlazada<String>
+ esFavorito(identificacionCliente: String, codigoInmueble: String): boolean
}

```

```

FavoritoDAO ..> ConexionDB : JDBC
@enduml

```

5. Capa Servicio

5.1 InventarioInmueblesService

```

@startuml InventarioInmueblesService
class InventarioInmueblesService {
- inventario: TablaHash<String, Inmueble>
- arbolPrecios: ArbolBinarioBusqueda<Double, Inmueble>
+ InventarioInmueblesService()
+ agregarInmueble(inmueble: Inmueble): void
+ buscarPorCodigo(codigo: String): Inmueble
+ obtenerOrdenadosPorPrecio(): ListaEnlazada<Inmueble>
+ obtenerTodos(): ListaEnlazada<Inmueble>
+ eliminarInmueble(codigo: String): void
}

```

```

InventarioInmueblesService *--> TablaHash
InventarioInmueblesService *--> ArbolBinarioBusqueda
InventarioInmueblesService ..> Inmueble
@enduml

```

5.2 ClientesService

```

@startuml ClientesService
class ClientesService {
- clientes: TablaHash<String, Cliente>
}

```

```

+ ClientesService()
+ agregarCliente(cliente: Cliente): void
+ buscarPorId(identificacion: String): Cliente
+ obtenerTodos(): ListaEnlazada<Cliente>
+ actualizar(cliente: Cliente): void
}

ClientesService *--> TablaHash
ClientesService ..> Cliente
@enduml

```

5.3 AuthService

```

@startuml AuthService
class AuthService {
- usuarioDAO: UsuarioDAO
- recomendacionService: RecomendacionService
+ AuthService(usuarioDAO, recomendacionService)
+ registrar(email, password, nombre, telefono): Map<String, Object>
+ autenticar(email, password): Usuario
+ crearAdmin(): void
+ simularEnvioRecomendaciones(email: String): ListaEnlazada<Map<String, Object>>
}

AuthService --> UsuarioDAO
AuthService --> RecomendacionService
AuthService ..> HashUtils : SHA-256
@enduml

```

5.4 OperacionesService

```

@startuml OperacionesService
class OperacionesService {
- operaciones: TablaHash<String, Operacion>
- historialOperaciones: ListaEnlazada<Operacion>
+ OperacionesService()
+ registrarOperacion(operacion: Operacion): void
+ buscarPorId(idOperacion: String): Operacion
+ obtenerHistorial(): ListaEnlazada<Operacion>
+ obtenerPorTipo(tipo: String): ListaEnlazada<Operacion>
+ cancelarOperacion(idOperacion: String): void
}

OperacionesService *--> TablaHash
OperacionesService *--> ListaEnlazada
OperacionesService ..> Operacion
@enduml

```

5.5 VisitaService

```

@startuml VisitaService
class VisitaService {
    - visitas: TablaHash<String, Visita>
    - visitaDAO: VisitaDAO
    + VisitaService(visitaDAO)
    + agendarVisita(visita: Visita, clienteDAO, inmuebleService): Visita
    + confirmarVisita(idVisita: String): void
    + realizarVisita(idVisita: String): void
    + cancelarVisita(idVisita: String): void
    + obtenerVisitasPorCliente(email: String): ListaEnlazada<Map<String, Object>>
    + obtenerVisitasPorAsesor(idAsesor: String): ListaEnlazada<Map<String, Object>>
    + tieneConflictoHorario(idAsesor: String, fechaHora: String): boolean
}

VisitaService *--> TablaHash
VisitaService --> VisitaDAO
VisitaService ..> Visita
@enduml

```

5.6 AlertasService

```

@startuml AlertasService
class AlertasService {
    - colaAlertas: ColaPrioridad<Alerta>
    + AlertasService()
    + generarAlertas(inventario, operaciones, visitas, clientes): void
    + siguienteAlerta(): Alerta
    + encolarAlerta(mensaje, prioridad): void
}

AlertasService *--> ColaPrioridad
AlertasService ..> Alerta
@enduml

```

5.7 GestorVisitasService

```

@startuml GestorVisitasService
class GestorVisitasService {
    - colaVisitas: ColaPrioridad<Visita>
    + GestorVisitasService()
    + encolarVisita(visita: Visita, prioridad: int): void
    + desencolarVisita(): Visita
    + obtenerSiguiete(): Visita
}

GestorVisitasService *--> ColaPrioridad
GestorVisitasService ..> Visita
@enduml

```


5.8 RecomendacionService

```
@startuml RecomendacionService
class RecomendacionService {
    + RecomendacionService()
    + recomendar(inmuebles: Inmueble[], cliente: Cliente): ListaEnlazada<Map<String, Object>>
    - calcularPuntaje(inmueble: Inmueble, cliente: Cliente): double
}

RecomendacionService ..> Inmueble
RecomendacionService ..> Cliente
RecomendacionService ..> ListaEnlazada : top 10
@enduml
```

5.9 AnalisisRelacionesService

```
@startuml AnalisisRelacionesService
class AnalisisRelacionesService {
    - grafoRelaciones: Grafo<String>
    + AnalisisRelacionesService()
    + registrarInteres(clienteId: String, inmuebleCodigo: String): void
    + recomendarRelacionados(inmuebleCodigo: String): ListaEnlazada<String>
}

AnalisisRelacionesService *--> Grafo
@enduml
```

5.10 HistorialCambiosService

```
@startuml HistorialCambiosService
class HistorialCambiosService {
    - pilaCambios: Pila<RegistroCambio>
    + HistorialCambiosService()
    + guardarCambio(codigoInmueble, campo, valorAnterior, valorNuevo): void
    + deshacerCambio(): RegistroCambio
    + obtenerHistorialCambios(): ListaEnlazada<String>
}

class RegistroCambio {
    - codigoInmueble: String
    - campo: String
    - valorAnterior: String
    - valorNuevo: String
}

HistorialCambiosService *--> Pila
HistorialCambiosService ..> RegistroCambio
@enduml
```

5.11 HistorialYFavoritosService

```
@startuml HistorialYFavoritosService
class HistorialYFavoritosService {
    + HistorialYFavoritosService()
    + agregarConsulta(cliente: Cliente, inmueble: Inmueble): void
    + toggleFavorito(cliente: Cliente, inmueble: Inmueble): void
    + obtenerFavoritos(cliente: Cliente): ListaEnlazada<Inmueble>
}

HistorialYFavoritosService ..> Cliente
HistorialYFavoritosService ..> Inmueble
@enduml
```

5.12 DetectorAnomaliesService

```
@startuml DetectorAnomaliesService
class DetectorAnomaliesService {
    + DetectorAnomaliesService()
    + detectarVisitasSinCierre(operaciones, visitas): ListaEnlazada<Map<String, Object>>
    + detectarSobrecargaAsesores(visitas): ListaEnlazada<Map<String, Object>>
    + detectarCambiosPrecioFrecuentes(historialCambios): ListaEnlazada<Map<String, Object>>
    + detectarConcentracionGeografica(visitas): ListaEnlazada<Map<String, Object>>
    + detectarTodas(operaciones, visitas, historialCambios): Map<String, Object>
}

DetectorAnomaliesService ..> ListaEnlazada : retorna
@enduml
```

5.13 ReporteService

```
@startuml ReporteService
class ReporteService {
    + ReporteService()
    + generarReporteRendimientoInmuebles(operaciones): Map<String, Object>
    + generarReporteAsesores(operaciones, asesores): ListaEnlazada<Map<String, Object>>
    + generarReportePreciosPorZona(inmuebles): ListaEnlazada<Map<String, Object>>
    + generarReporteClientesActivos(clientes, visitas, operaciones): ListaEnlazada<Map<String, Object>>
    + generarReporteTiposOperacion(operaciones): Map<String, Object>
    + generarReporteFiltrado(operaciones, tipo, zona, precioMin, precioMax):
ListaEnlazada<Map<String, Object>>
}

ReporteService ..> ListaEnlazada : retorna
@enduml
```

6. Main + Utilidades

6.1 Main

```
@startuml Main
class Main {
    + {static} main(args: String[]): void
    - {static} fechasConConflicto(fecha1, fecha2): boolean
}

Main ..> "modelo.*"
Main ..> "dao.*"
Main ..> "servicio.*"
Main ..> "utilidades.*"
Main --> ConexionDB : inicializarTablas()
Main --> AuthService : crearAdmin()
Main --> AlertasService : generarAlertas()
Main ..> Javalin : API REST (50+ endpoints)
@enduml
```

6.2 HashUtils

```
@startuml HashUtils
class HashUtils {
    + {static} hashPassword(password: String): String
}

HashUtils ..> "java.security.MessageDigest" : SHA-256
@enduml
```

7. Diagrama de Relaciones Completo

Este es el diagrama maestro que muestra **todas las relaciones** entre paquetes, clases y dependencias.

```
@startuml DiagramaCompleto
skinparam packageStyle rectangle
skinparam shadowing false

' ===== MODELO =====
package "com.proptech.modelo" as MODELO {
    class Inmueble
    class Cliente {
        historialConsultas: ListaEnlazada<Inmueble>
        favoritos: ListaEnlazada<Inmueble>
    }
    class Asesor
    class Visita {
        + cliente: Cliente
    }
}
```

```

    + inmueble: Inmueble
    + asesor: Asesor
}
class Operacion {
    + inmueble: Inmueble
    + cliente: Cliente
    + asesor: Asesor
}
class Alerta
class Rol {
    + CLIENTE, VENDEDOR, ADMIN, GERENTE
}
class Usuario {
    + rol: Rol
}
Visita *--> Cliente
Visita *--> Inmueble
Visita *--> Asesor
Operacion *--> Inmueble
Operacion *--> Cliente
Operacion *--> Asesor
Usuario *--> Rol
Cliente ..> Inmueble : "ListaEnlazada"
}

' ===== ESTRUCTURAS =====
package "com.proptech.utilidades.estructuras" as ESTRUCTURAS {
    class Nodo<T>
    class ListaEnlazada<T>
    class Pila<T>
    class Cola<T>
    class NodoPrioridad<T>
    class ColaPrioridad<T>
    class NodoArbol<K, V>
    class ArbolBinarioBusqueda<K, V>
    class EntradaHash<K, V>
    class TablaHash<K, V>
    class Vertice<T>
    class Grafo<T>

    ListaEnlazada o--> Nodo
    Pila o--> Nodo
    Cola o--> Nodo : frente + final
    ColaPrioridad o--> NodoPrioridad
    ArbolBinarioBusqueda o--> NodoArbol
    TablaHash o--> EntradaHash
    TablaHash ..> ListaEnlazada : valores/keys
    Grafo o--> Vertice
    Vertice ..> ListaEnlazada : adyacentes
}

' ===== UTILIDADES =====

```

```

package "com.proptech.utilidades" as UTIL {
    class HashUtils
}

' ===== DAO =====
package "com.proptech.dao" as DAO {
    class ConexionDB
    class InmuebleDAO
    class ClienteDAO
    class AsesorDAO
    class UsuarioDAO {
        - cache: ConcurrentHashMap
    }
    class OperacionDAO
    class VisitaDAO
    class FavoritoDAO

    InmuebleDAO ..> MODELO.Inmueble : CRUD
    ClienteDAO ..> MODELO.Cliente : CRUD
    AsesorDAO ..> MODELO.Asesor : CRUD
    UsuarioDAO ..> MODELO.Usuario : CRUD + cache
    UsuarioDAO ..> MODELO.Rol
    OperacionDAO ..> MODELO.Operacion : CRUD
    VisitaDAO ..> MODELO.Visita : CRUD
    FavoritoDAO --> MODELO.Cliente
    FavoritoDAO --> MODELO.Inmueble
    DAO ..> ConexionDB : JDBC
    DAO ..> ESTRUCTURAS.ListaEnlazada : retorna
}

' ===== SERVICIO =====
package "com.proptech.servicio" as SERVICIO {
    class InventarioInmueblesService {
        - inventario: TablaHash
        - arbolPrecios: ArbolBinarioBusqueda
    }
    class ClientesService {
        - clientes: TablaHash
    }
    class AuthService
    class OperacionesService {
        - operaciones: TablaHash
        - historial: ListaEnlazada
    }
    class VisitaService {
        - visitas: TablaHash
    }
    class AlertasService {
        - colaAlertas: ColaPrioridad
    }
    class GestorVisitasService {
        - colaVisitas: ColaPrioridad
    }
}

```

```

}
class RecomendacionService
class AnalisisRelacionesService {
    - grafoRelaciones: Grafo
}
class HistorialCambiosService {
    - pilaCambios: Pila
}
class HistorialYFavoritosService
class DetectorAnomaliesService
class ReporteService

' Servicio -> Estructuras
InventarioInmueblesService *--> ESTRUCTURAS.TablaHash
InventarioInmueblesService *--> ESTRUCTURAS.ArbolBinarioBusqueda
ClientesService *--> ESTRUCTURAS.TablaHash
OperacionesService *--> ESTRUCTURAS.TablaHash
OperacionesService *--> ESTRUCTURAS.ListaEnlazada
VisitaService *--> ESTRUCTURAS.TablaHash
AlertasService *--> ESTRUCTURAS.ColaPrioridad
GestorVisitasService *--> ESTRUCTURAS.ColaPrioridad
AnalisisRelacionesService *--> ESTRUCTURAS.Grafo
HistorialCambiosService *--> ESTRUCTURAS.Pila

' Servicio -> Modelo
InventarioInmueblesService ..> MODELO.Inmueble
ClientesService ..> MODELO.Cliente
OperacionesService ..> MODELO Operacion
VisitaService ..> MODELO.Visita
AlertasService ..> MODELO.Alerta
RecomendacionService ..> MODELO.Inmueble
RecomendacionService ..> MODELO.Cliente
HistorialYFavoritosService ..> MODELO.Cliente
HistorialYFavoritosService ..> MODELO.Inmueble
AuthService ..> MODELO.Usuario

' Servicio -> DAO
VisitaService --> DAO.VisitaDAO
AuthService --> DAO.UsuarioDAO

' Servicio -> Servicio
AuthService --> RecomendacionService
}

' ===== MAIN =====
package "com.proptech" as MAIN {
    class Main_stub as "Main" {
        + {static} main(args: String[]): void
        - {static} fechasConConflicto(): boolean
    }
}

```

```
' Relaciones desde Main
MAIN.Main_stub ..> MODELO
MAIN.Main_stub ..> DAO : inicializa
MAIN.Main_stub ..> SERVICIO : inicializa
MAIN.Main_stub ..> ESTRUCTURAS
MAIN.Main_stub ..> UTIL.HashUtils

@enduml
```

Cómo usar estos diagramas

1. Copia cualquier bloque `@startuml ... @enduml` en:

- **PlantUML Online:** <https://www.plantuml.com/plantuml/uml/>
- **VS Code:** Extensión "PlantUML" (jerezadev / qjebbs)
- **draw.io:** Insert → Advanced → PlantUML
- **IntelliJ IDEA:** Plugin "PlantUML integration"

2. Para generar imágenes:

```
# Si tienes PlantUML instalado (requiere Java)
java -jar plantuml.jar diagrama.txt

# O con Node.js
npx puml-for-markdown diagrama.md
```